Department of Computer Engineering
CS-302 Artificial Intelligence

Experiment # 6
Understanding and implementing Artificial Neural Network

Objective/s:

- Overview of Neural Network
- Working of ANN
- Example
- Lab Tasks

1. Overview of Neural Network

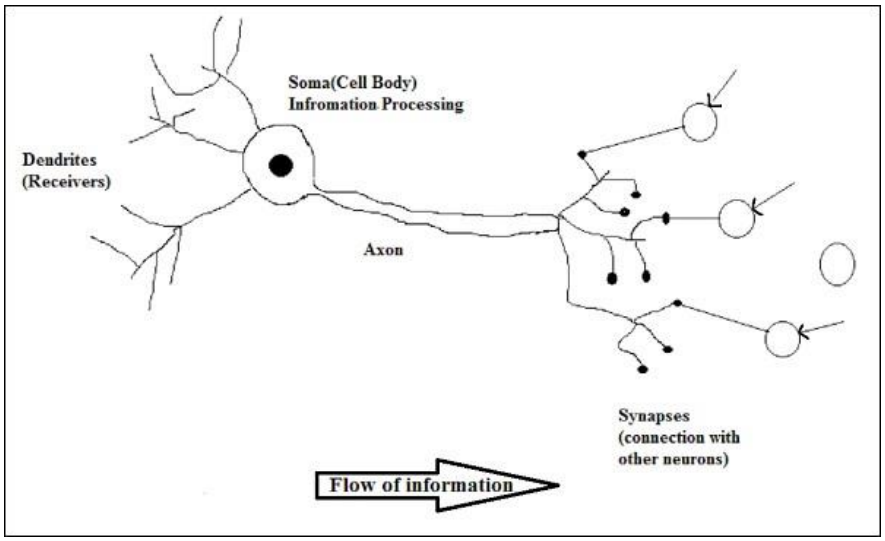
Artificial Neural Network (ANN) is an efficient computing system whose central theme is borrowed from the analogy of biological neural networks. ANNs are also named as "artificial neural systems," or "parallel distributed processing systems," or "connectionist systems." ANN acquires a large collection of units that are interconnected in some pattern to allow communication between the units. These units, also referred to as nodes or neurons, are simple processors which operate in parallel.

Every neuron relates to other neuron through a connection link. Each connection link is associated with a weight that has information about the input signal. This is the most useful information for neurons to solve a particular problem because the weight usually excites or inhibits the signal that is being communicated. Each neuron has an internal state, which is called an activation signal. Output signals, which are produced after combining the input signals and activation rule, may be sent to other units.

1.1 Biological Neuron

A nerve cell neuron is a special biological cell that processes information. According to an estimation, there are huge number of neurons, approximately 10^{11} with numerous interconnections, approximately 10^{15}.

1.1.1 Schematic Diagram



1.2 Working of a Biological Neuron

As shown in the above diagram, a typical neuron consists of the following four parts with the help of which we can explain its working

- **Dendrites** − They are tree-like branches, responsible for receiving the information from other neurons it is connected to. In other sense, we can say that they are like the ears of neuron.
- **Soma** − It is the cell body of the neuron and is responsible for processing of information, they have received from dendrites.
- **Axon** − It is just like a cable through which neurons send the information.
- **Synapses** − It is the connection between the axon and other neuron dendrites.

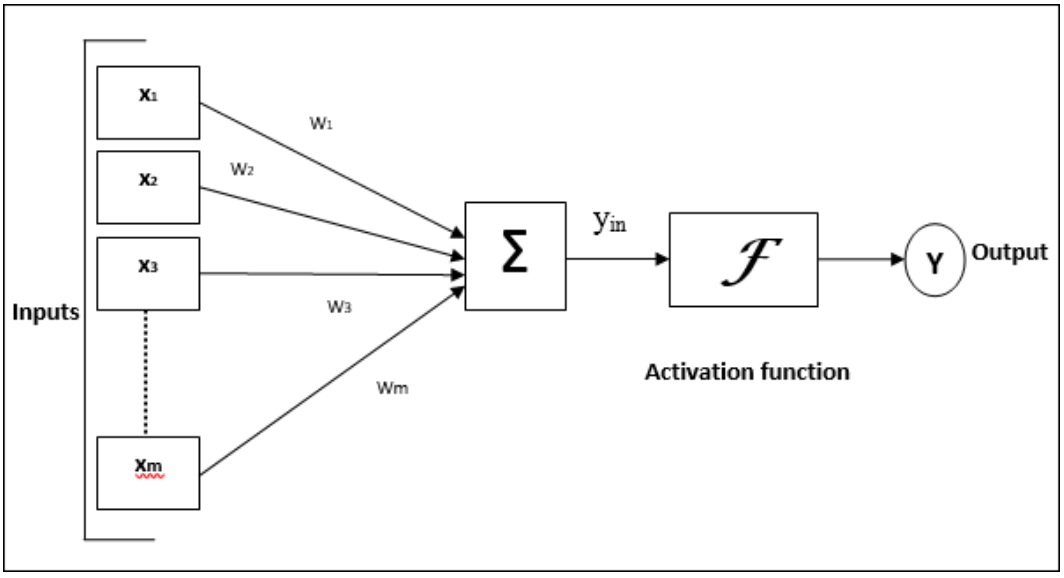
1.3 BNN versus ANN

Before looking at the differences between Artificial Neural Network and Biological Neural Network, let us take a look at the similarities based on the terminology between these two.

Biological Neural Network BNN	**Artificial Neural Network** ANN
Soma	Node
Dendrites	Input
Synapse	Weights or Interconnections
Axon	Output

2. Model of Artificial Neural Network

The following diagram represents the general model of ANN followed by its processing.



For the above general model of artificial neural network, the net input can be calculated as follows −

$$y_{in} = x_1 . w_1 + x_2 . w_2 + x_3 . w_3 \ldots x_m . w_m$$

$$\text{i.e., Net input } y_{in} = \sum_i^m x_i . w_i$$

The output can be calculated by applying the activation function over the net input.

$$Y = F(y_{in})$$

Example

```
# Import necessary libraries
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from sklearn.datasets import load_iris
from sklearn.metrics import accuracy_score

# Step 1: Load the Iris dataset
iris = load_iris()
X = iris.data  # Features (4 features per sample)
y = iris.target  # Target classes (3 classes)

# Step 2: Preprocess the data (use only two classes for binary classification)
# We will classify class 0 (Setosa) vs class 1 (Versicolor)
X = X[y != 2]  # Remove class 2 (Virginica)
y = y[y != 2]  # Remove class 2 (Virginica)

# Step 3: Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Step 4: Standardize the data (helpful for ANN performance)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Step 5: Build the ANN model
model = Sequential()

# Input layer and first hidden layer with 8 neurons and ReLU activation
model.add(Dense(units=8, input_dim=X_train.shape[1], activation='relu'))

# Output layer with 1 neuron and sigmoid activation (for binary classification)
model.add(Dense(units=1, activation='sigmoid'))

# Step 6: Compile the model
model.compile(loss='binary_crossentropy',  # For binary classification
              optimizer='adam',  # Optimizer
              metrics=['accuracy'])  # Evaluation metric

# Step 7: Train the model
history = model.fit(X_train, y_train, epochs=50, batch_size=10, validation_split=0.2)

# Step 8: Evaluate the model
y_pred_prob = model.predict(X_test)
y_pred = (y_pred_prob > 0.5).astype(int)  # Convert probabilities to binary predictions
```

```
# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)
print(f"Model Accuracy: {accuracy * 100:.2f}%")

# Optional: Plotting training history (loss and accuracy)
import matplotlib.pyplot as plt

plt.figure(figsize=(12, 6))

# Plotting loss
plt.subplot(1, 2, 1)
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Loss during Training')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()

# Plotting accuracy
plt.subplot(1, 2, 2)
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title('Accuracy during Training')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()

plt.show()
```

3. Tasks

Build a Simple ANN for Binary Classification

- **Instructions**:
  - Use a binary classification dataset (e.g., **Iris dataset**, classify Setosa vs. Versicolor).
  - Build an ANN with:
    - **Input Layer**: 4 neurons (one for each feature).
    - **Hidden Layer**: 8 neurons with **ReLU activation**.
    - **Output Layer**: 1 neuron with **sigmoid activation** for binary classification.
  - Use **binary cross-entropy** as the loss function and **adam optimizer**.